

Week16_예습과제

통계 수정 삭제

카사바 · 방금 전

 0

Euron_2026-1



▼ 목록 보기

9/9



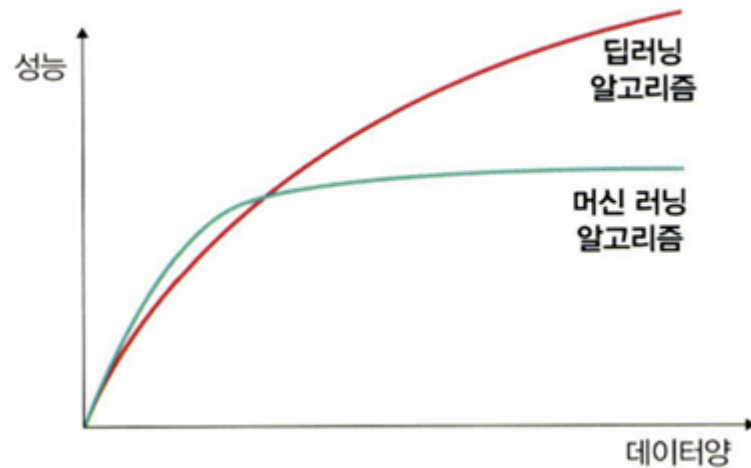
8장 성능 최적화

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

- 최대한 많은 데이터 수집하기: 많은 데이터를 수집한다. 일반적으로 데이터양이 많을수록 성능이 좋기 때문이다.

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 데이터 생성하기: 많은 데이터를 수집할 수 없다면 데이터를 만들어 사용한다.
- 데이터 범위 조정하기: 활성화 함수로 시그모이드를 사용한다면 데이터셋 범위를 0~1로 설정하고, 하이퍼볼릭 탄젠트를 사용한다면 -1~1로 설정한다.
- 정규화, 규제화, 표준화도 도움이 된다.

8.1.2 알고리즘을 이용한 성능 최적화

유사한 용도의 알고리즘들로 모델을 훈련시켜 보고 최적의 성능을 보이는 알고리즘을 선택해야 한다.

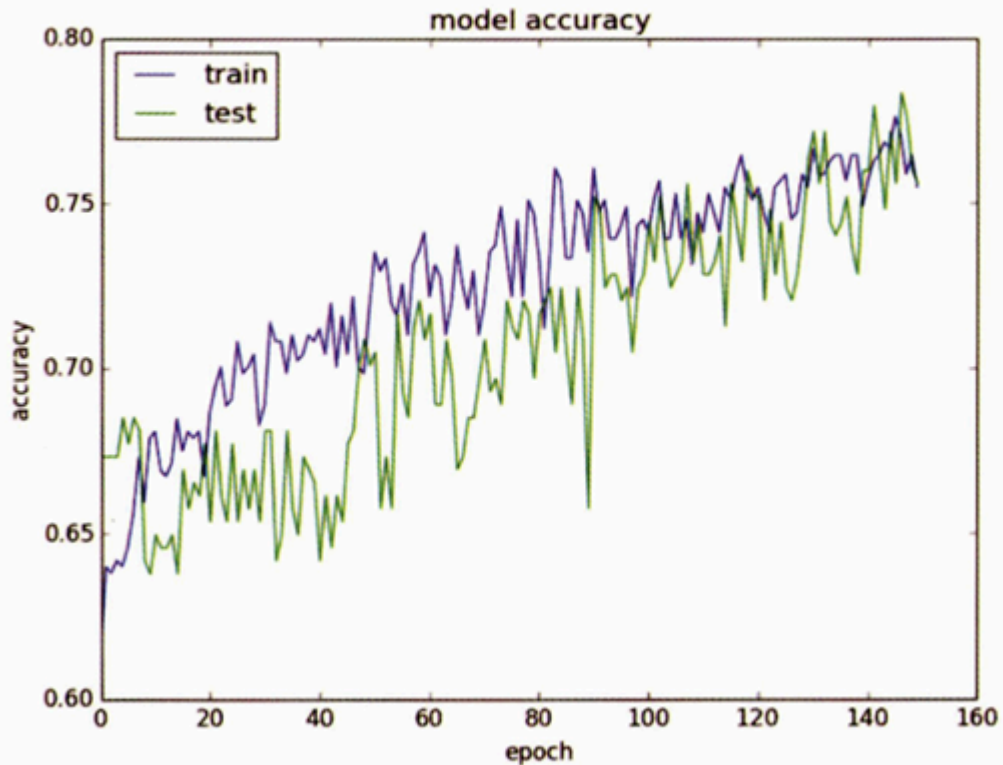
e.g. 머신러닝에서 데이터 분류를 위해 SVM, K-최근접 이웃 알고리즘 등 선택하여 훈련, 시계열 데이터의 경우 RNN, LSTM, GRU 등의 알고리즘을 훈련시켜 성능이 가장 좋은 모델 선택하여 사용

8.1.3 알고리즘 튜닝을 위한 성능 최적화

성능 최적화를 하는 데 가장 많은 시간이 소요되는 부분. 다양한 하이퍼파라미터를 변경하면서 훈련시키고 최적의 성능을 도출해야 한다.

- 진단: 모델 평가 결과를 바탕으로 모델이 과적합되었는지, 혹은 다른 원인이 있는지에 대한 인사이트를 얻을 수 있다. 예를 들어 훈련 성능이 검증 성능보다 눈에 띄게 좋으면 과적합을 의심할 수 있다. 그리고 이를 해결하기 위해 규제화를 진행한다면 성능 향상에 도움이 된다. 또 훈련과 검증 결과가 모두 좋지 않다면 과소적합을 의심할 수 있다. 이때는 네트워크 구조를 변경하거나 훈련을 늘리기 위해 에포크 수를 조정해 볼 수 있다. 그리고 훈련 성능이 검증 성능을 넘어서는 변곡점이 있다면 조기 종료를 고려할 수 있다.

▼ 그림 8-2 알고리즘 성능 진단



- 가중치: 초깃값은 작은 난수를 사용한다. 아니면 오토인코더 같은 비지도 학습을 이용하여 사전 훈련을 진행한 후 지도 학습을 진행할 수도 있다.
- 학습률: 모델의 네트워크 구성에 따라 다르기 때문에 초기에 매우 크거나 작은 임의의 난수를 선택하여 학습 결과를 보고 조금씩 변경해야 한다. 네트워크 계층이 많다면 학습률을 높여야 하고, 적다면 작게 설정해야 한다.
- 활성화 함수: 활성화 함수를 변경할 때 손실 함수도 함께 변경해야 하는 경우가 많다. 그래서 변경할 때 신중해야 한다. 일반적으로 활성화 함수로 시그모이드나 하이퍼볼릭 탄젠트를 사용했다면, 출력층에서는 소프트맥스나 시그모이드를 많이 선택한다.
- 배치와 에포크: 최근 딥러닝 트렌드는 큰 에포크와 작은 배치를 사용하는 것이다. 하지만 적절한 배치 크기를 위해 훈련 데이터셋 크기와 똑같이 설정하거나, 하나의 배치로 훈련시켜 보는 등 다양한 테스트를 진행하는 것이 좋다.
- 옵티마이저 및 손실 함수: 일반적으로 옵티마이저는 확률적 경사 하강법을 많이 사용한다. 아담이나 알엠에스프롭도 좋은 성능을 보인다.
- 네트워크 구성(네트워크 토폴로지): 네트워크 구성을 변경해 가면서 성능을 테스트해야 한다. 예를 들어 하나의 은닉층에 뉴런 여러 개를 포함시키거나(네트워크가 넓다고 표현),

네트워크 계층을 늘리되 뉴런 개수를 줄여 본다(네트워크가 깊다고 표현). 혹은 두 가지를 결합한다.

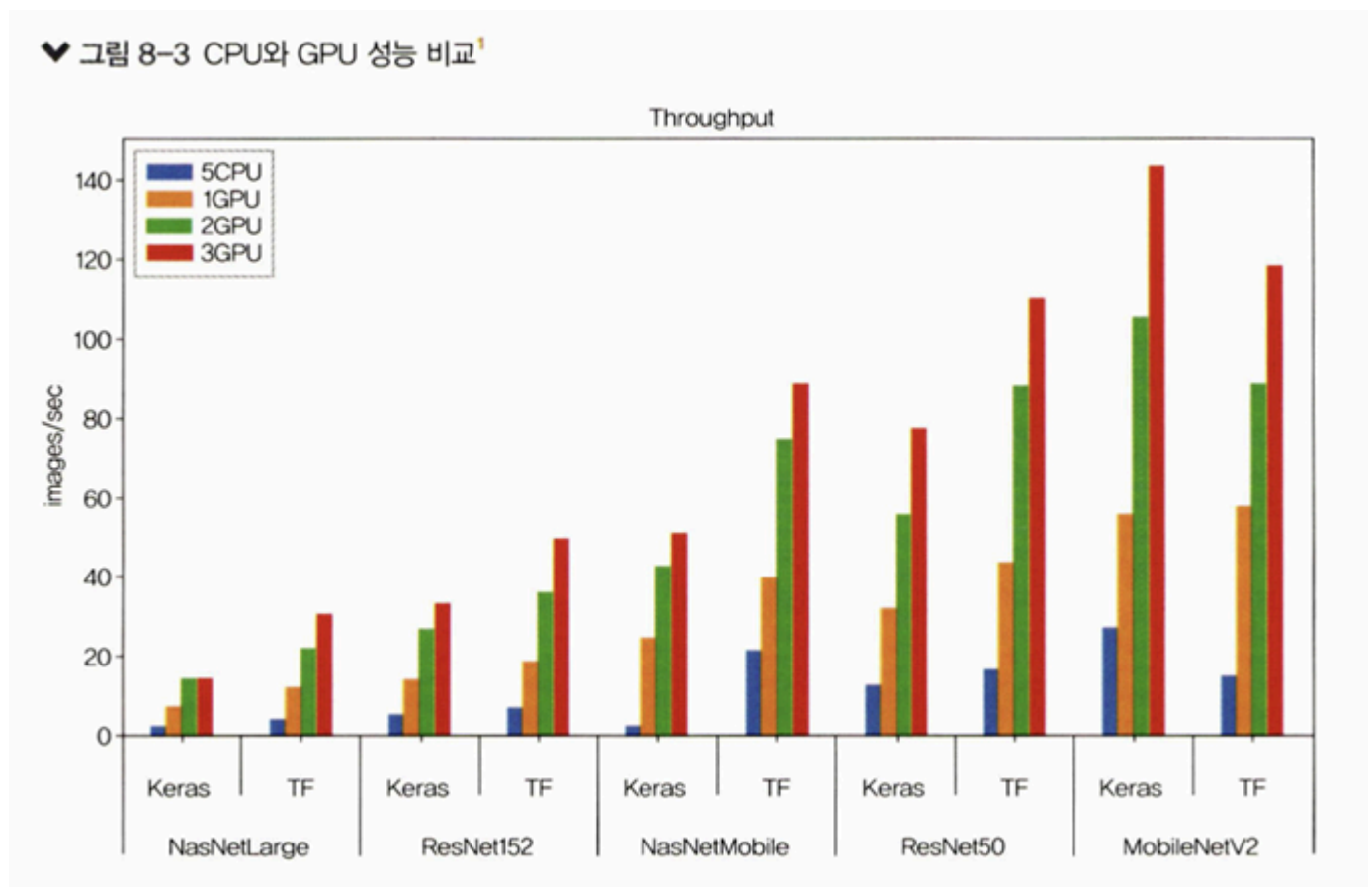
8.1.4 앙상블을 이용한 성능 최적화

앙상블: 모델을 두 개 이상 섞어 사용하는 것

8.2 하드웨어를 이용한 성능 최적화

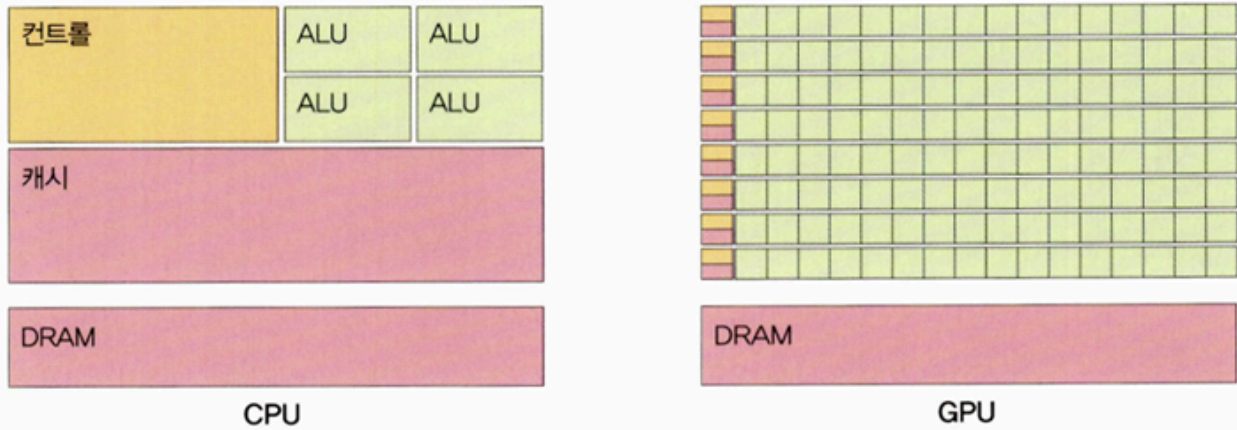
CPU가 아닌 GPU를 이용하는 것

8.2.1 CPU와 GPU 사용의 차이



그래프를 보면 CPU 다섯 개를 동시에 돌려도 GPU 한 개보다 성능이 좋지 못한 것을 확인할 수 있다.

▼ 그림 8-4 CPU와 GPU의 구조 비교



CPU: 연산을 담당하는 ALU, 명령어를 해석하고 실행하는 컨트롤, 데이터를 담아 두는 캐시로 구성됨. 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식. 즉, 한 번에 하나의 명령어만 처리하기 때문에 연산을 담당하는 ALU가 많을 필요가 없다.

GPU: 병렬 처리를 위해 개발됨. 캐시 메모리 비중은 낮고, 연산을 수행하는 ALU 개수가 많아짐. 서로 다른 명령어를 동시에 병렬적으로 처리하도록 설계되었기 때문에 성능에 부담이 없다. 또, 하나의 코어에 ALU가 수백~수천 개 장착되어 있어서 CPU로는 시간이 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행할 수 있다.

개별 코어 속도는 CPU가 GPU보다 훨씬 빠르다.

파이썬이나 매트랩처럼 행렬 연산을 많이 사용하는 재귀 연산이 대표적인 직렬 연산을 수행한다. 즉, 3x3 행렬에서 ABC 열이 있을 때 A열이 처리된 후에야 B열이 처리되고 C열이 처리되는 순차적 연산에서는 CPU가 적합하다.

하지만 역전파처럼 복잡한 미적분은 병렬 연산을 해야 속도가 빨라진다. 이와 같은 병렬 처리는 복잡한 연산이 수반되는 딥러닝에서 속도와 성능을 높여 주는 주요 요인이 될 수 있다. 따라서 딥러닝에서 GPU 사용은 필수라고 할 수 있다.

8.2.2 GPU를 이용한 성능 최적화

윈도우 환경에서 GPU용 파이토치를 설치하려면 CUDA와 cuDNN을 설치해야 한다.

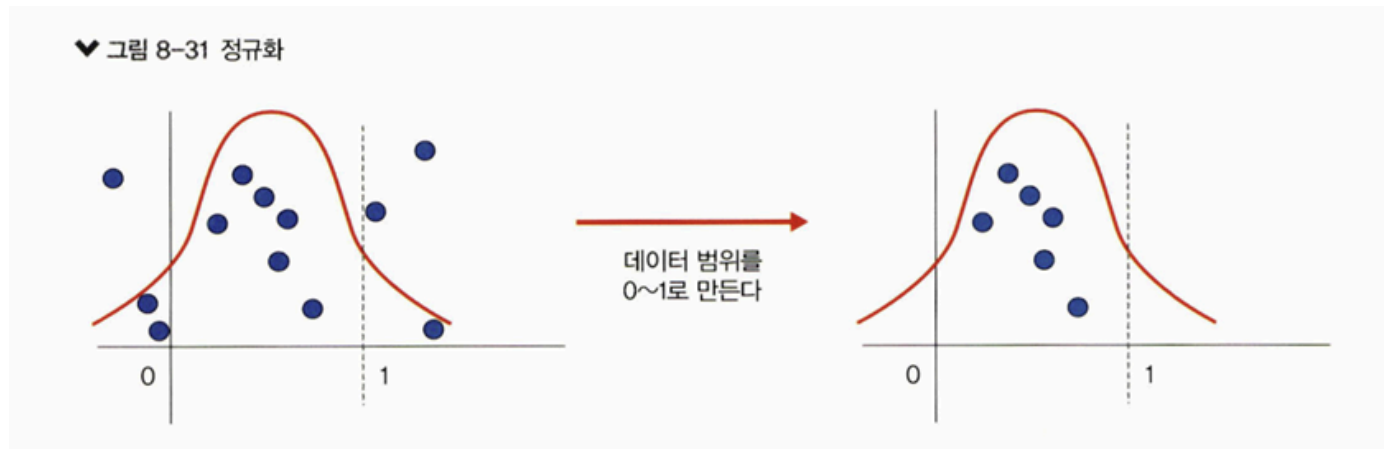
CUDA: NVIDIA에서 개발한 GPU 개발 툴. CUDA를 사용하면 많은 양의 연산을 동시에 처리할 수 있다.

8.3 하이퍼파라미터를 이용한 성능 최적화

8.3.1 배치 정규화를 이용한 성능 최적화

정규화

정규화: 데이터 범위를 사용자가 원하는 범위로 제한하는 것. 예를 들어 이미지 데이터는 픽셀 정보로 0~255 사이의 값을 갖는데, 이를 255로 나누면 0~1.0 사이 값을 갖게 된다.



각 특성 범위를 조정한다는 의미로 특성 스케일링이라고도 한다.

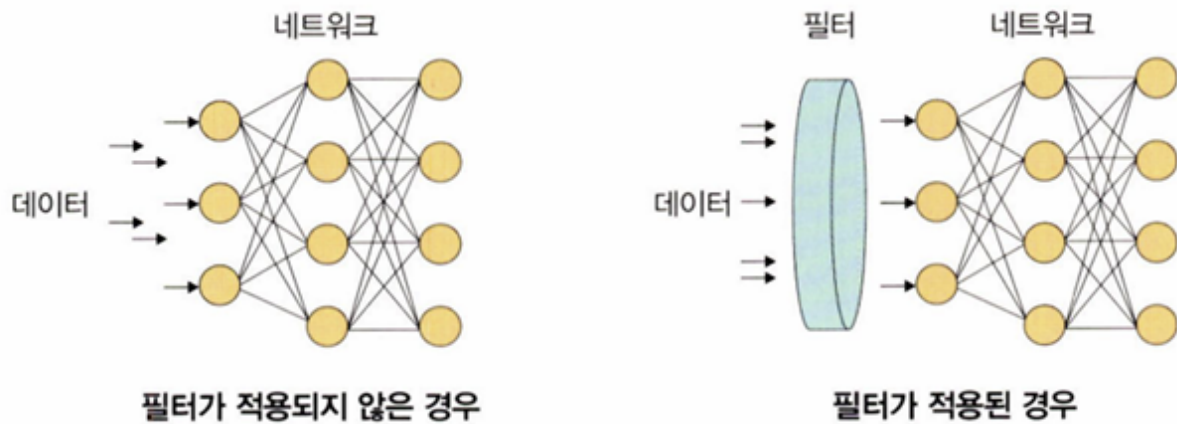
$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x : 입력 데이터)

규제화

규제화: 모델 복잡도를 줄이기 위해 제약을 두는 방법. 이때 제약은 데이터가 네트워크에 들어가기 전에 필터를 적용하는 것.

▼ 그림 8-32 규제화

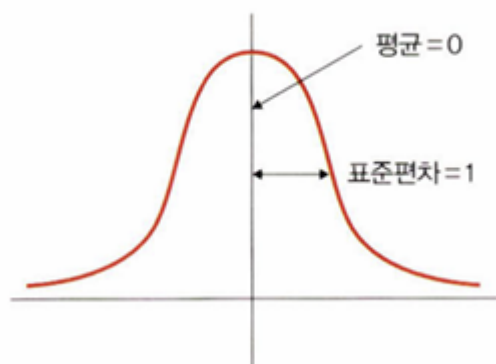


왼쪽 그림은 필터가 적용되지 않은 경우로, 모든 데이터가 네트워크에 투입되지만, 오른쪽 그림에서는 필터로 걸러진 데이터만 네트워크에 투입되어 빠르고 정확한 결과를 얻을 수 있다. 규제를 이용하여 모델 복잡도를 줄이는 방법으로는 드롭아웃, 조기 종료 등이 있다.

표준화

표준화: 기존 데이터를 평균 0, 표준편차 1 형태의 데이터로 만드는 방법. 표준화 스칼라 혹은 z-스코어 정규화라고도 한다.

▼ 그림 8-33 표준화



$$\frac{x - m}{\sigma}$$

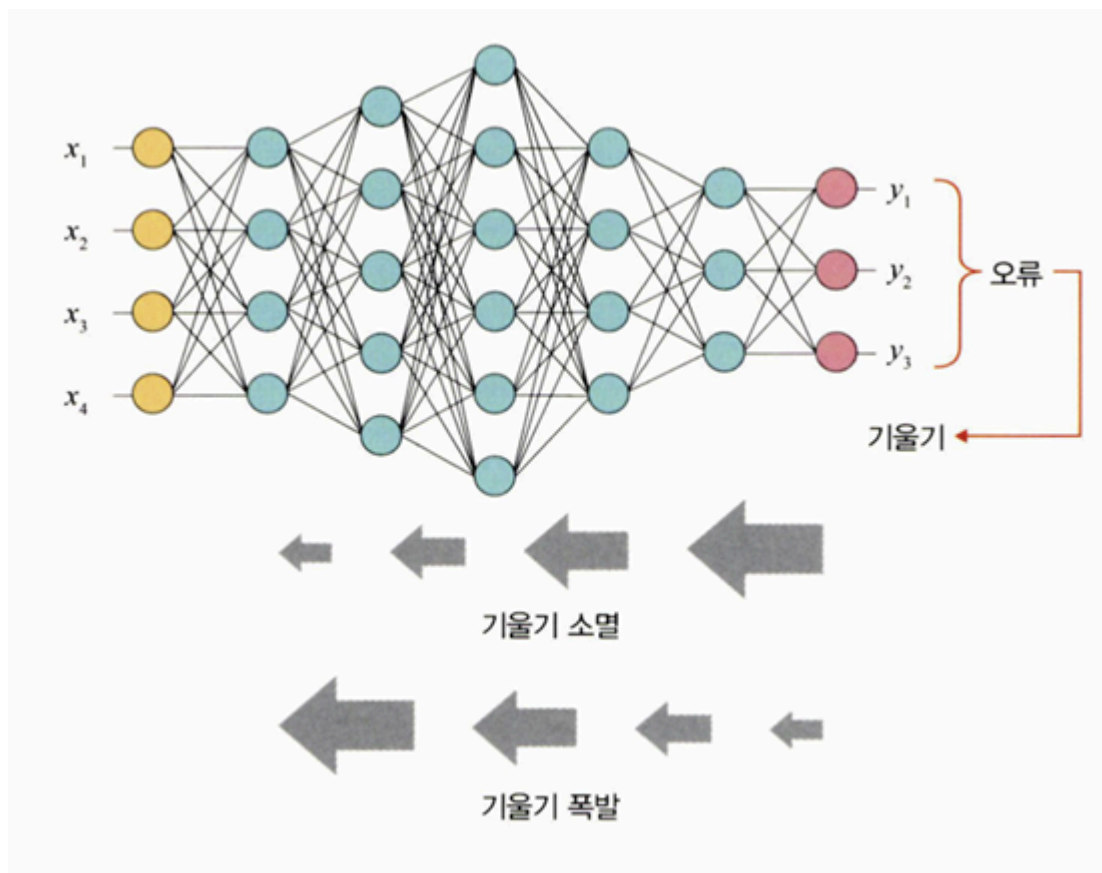
(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

배치 정규화

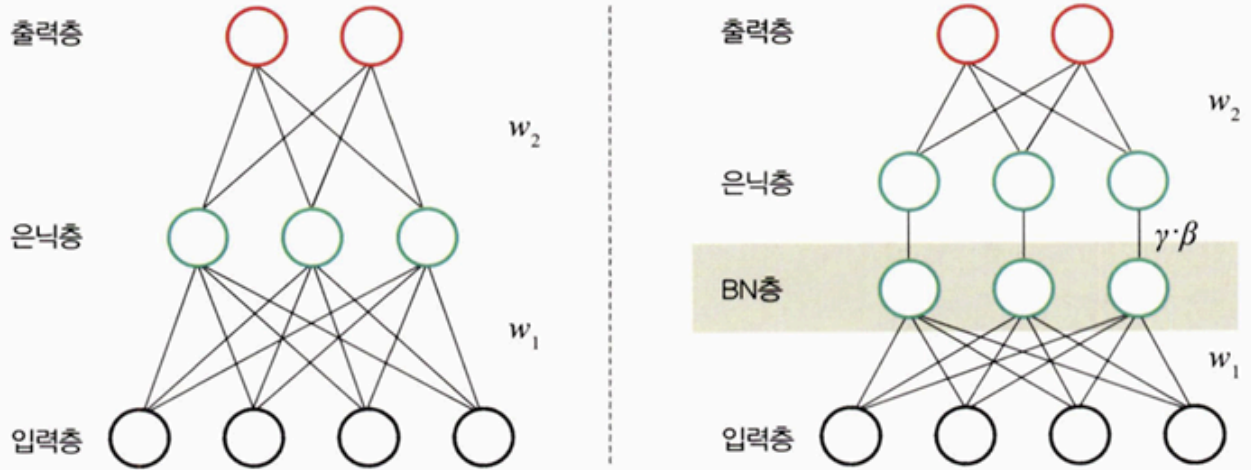
배치 정규화: 데이터 분포를 안정시켜 학습 속도를 높일 수 있다. 기울기 소멸, 기울기 폭발 같은 문제를 해결할 때 쓰인다. 일반적으로 기울기 소멸이나 폭발 문제를 해결하기 위해 손실 함수로 ReLU를 사용하거나 초깃값 튜닝, 학습률 등을 조정한다.

기울기 소멸: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상

기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상



♥ 그림 8-35 배치 정규화



기울기 소멸과 폭발 원인은 내부 공변량 변화 때문이다.

내부 공변량 변화: 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상

따라서 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 미니 배치에 적용해 평균은 0으로, 표준편차는 1로 유지하도록 한다.

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{---- ①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{---- ②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{---- ③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{---- ④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

1. 미니 배치 평균을 구한다.
2. 미니 배치의 분산과 표준편차를 구한다.
3. 정규화를 수행한다.
4. 스케일을 조정한다.

매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정해지기 때문에 속도를 향상시킬 수 있지만 단점도 있다.

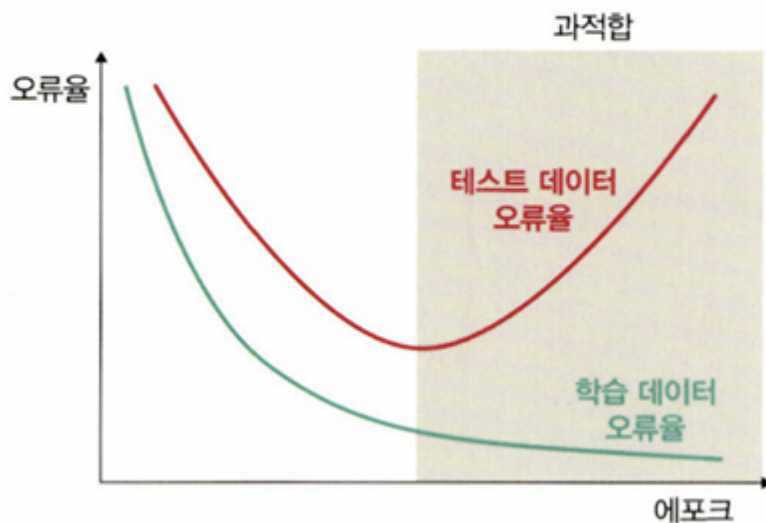
첫째, 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있다. 예를 들어 분산이 0이면 정규화 자체가 안 될 수도 있다.

둘째, RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 모델이 더 복잡해지면서 비효율적일 수 있다.

8.3.2 드롭아웃을 이용한 성능 최적화

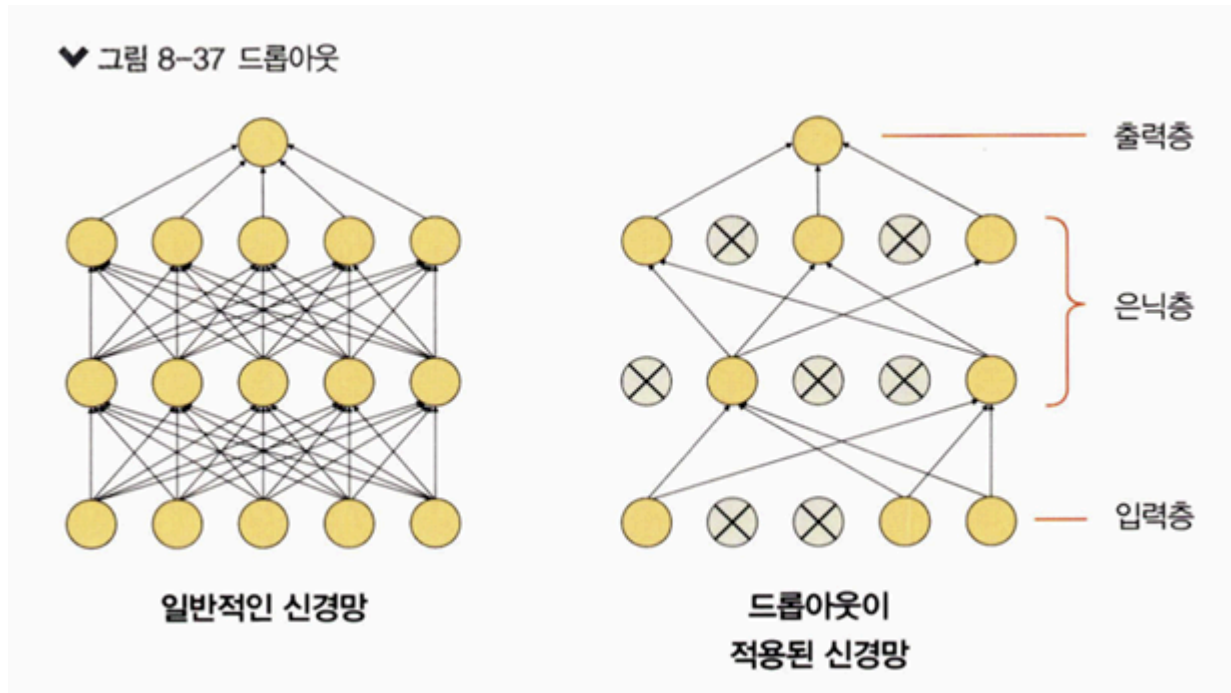
일반적으로 훈련 데이터셋은 실제 데이터셋의 부분 집합이므로 훈련 데이터셋에 대해서는 오류가 감소하지만, 테스트 데이터셋에 대해서는 오류가 증가한다. 즉, 훈련 데이터셋에 대해 훈련을 계속한다면 오류는 줄어들지만 테스트 데이터셋에 대한 오류는 어느 순간부터 증가하는데, 이러한 모델을 과적합되어 있다고 한다.

▼ 그림 8-36 훈련과 오류율



드롭아웃: 훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 해당하는 가중치는

업데이트하지 않는 방법. 매 단계마다 사용하지 않는 뉴런을 바꾸어 가며 훈련시킨다. 꺼진 노드는 신호를 전달하지 않으므로 지나친 학습을 방지하는 효과가 생긴다.



어떤 노드를 비활성화할지는 학습할 때마다 무작위로 선정되며, 테스트 데이터로 평가할 때는 노드들을 모두 사용하여 출력하되 노드 삭제 비율(드롭아웃 비율)을 곱해서 성능을 평가한다.

드롭아웃을 사용하면 훈련 시간이 길어지는 단점이 있지만, 모델 성능을 향상하기 위해 자주 쓴다.

코드 8-1 라이브러리 호출

```
import torch
import matplotlib.pyplot as plt
import numpy as np

import torchvision
import torchvision.transforms as transforms

import torch.nn as nn
import torch.optim as optim
```

예제에서 사용할 FashionMNIST 데이터셋을 내려받는다.

코드 8-2 데이터셋 내려받기

```
trainset = torchvision.datasets.FashionMNIST(
    root='../chap08/data/', train=True,
```

```

download=True,
transform=transforms.ToTensor()
)

```

코드 8-3 데이터셋을 메모리로 가져오기

```

batch_size = 4
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)

```

데이터셋을 이미지와 레이블로 분리하여 학습을 위한 준비를 한다. 분리된 데이터셋에 대한 정보를 확인하자.

코드 8-4 데이터셋 분리

```

dataiter = iter(trainloader)
images, labels = next(dataiter)

print(images.shape)
print(images[0].shape)
print(labels[0].item())

```

`torch.Size([4, 1, 28, 28])`

(a) (b) (c)

- a. 한 번의 배치 크기로 몇 개의 데이터를 가져오는지 의미한다. `batch_size = 4`로 설정했기 때문에 4를 출력한다.
- b. 채널. 흑백 이미지는 1, 컬러 이미지는 3
- c. 28x28(너비x높이) 픽셀 크기 이미지

코드 8-5 이미지 데이터를 출력하기 위한 전처리

```

def imshow(img, title):
    plt.figure(figsize=(batch_size * 4, 4)) # 출력할 개별 이미지의 크기 지정
    plt.axis('off')
    plt.imshow(np.transpose(img, (1, 2, 0))) # 1.
    plt.title(title)
    plt.show()

```

1. 파이토치는 이미지 데이터셋을 [배치 크기, 채널, 너비, 높이] 순서대로 저장한다. 하지만 이를 맷플롯립으로 출력하기 위해서는 이미지가 [너비, 높이, 채널] 형태여야 한다. 즉, 데이터 형태 변경이 필요한데, 이때 사용 가능한 것이 넘파이 라이브러리의 `transpose()`

코드 8-6 이미지 데이터 출력 함수

```
def show_batch_images(dataloader):
    images, labels = next(iter(dataloader)) # 이미지 크기는 (4,28,28,1)이 된다. (배치 크기, 높이, 너비, 채널)

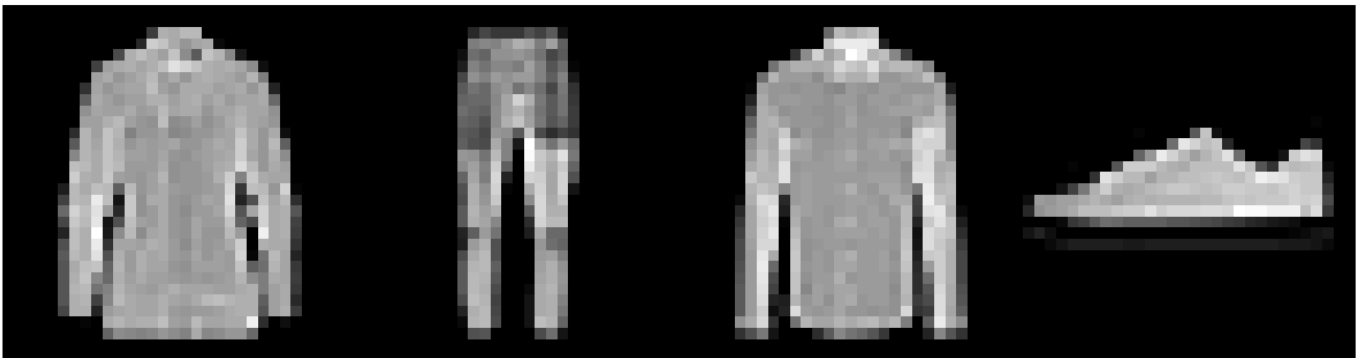
    img = torchvision.utils.make_grid(images) # 좌표에 이미지 픽셀을 대응시켜 그리드 형태로 출력
    imshow(img, title=[str(x.item()) for x in labels]) # imshow 함수를 사용함으로써 데이터의 형태가 (

    return images, labels
```

코드 8-7 이미지 출력

```
images, labels = show_batch_images(trainloader)
```

['4', '1', '6', '7']



```
classes = {
    0: "T-Shirt/Top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot"
}
```

하나의 배치에서 네 개의 이미지만 가져오도록 해서 네 개의 이미지가 출력된다.

코드 8-8 배치 정규화가 적용되지 않은 네트워크

```
class NormalNet(nn.Module):
    def __init__(self):
        super(NormalNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), # (28,28) 크기의 이미지로 입력은 784(28x28) 크기가 된다.
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.ReLU(),
            nn.Linear(24, 10) # FashionMNIST 클래스 10개
        ) # nn.Sequential을 사용하면 forward() 함수에서 계층별로 가독성 있게 코드 구현 가능

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x) # nn.Sequential에서 정의한 계층 호출
        return x
```

코드 8-9 배치 정규화가 포함된 네트워크

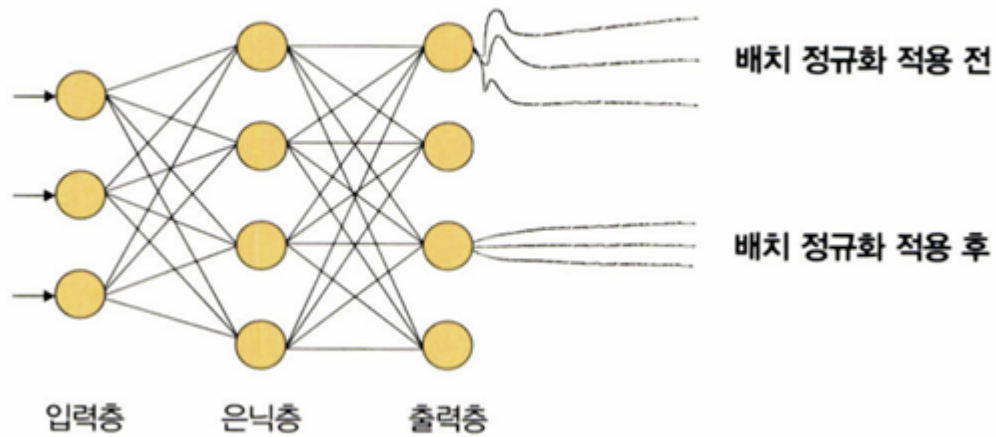
```
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.BatchNorm1d(48), # 1.
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

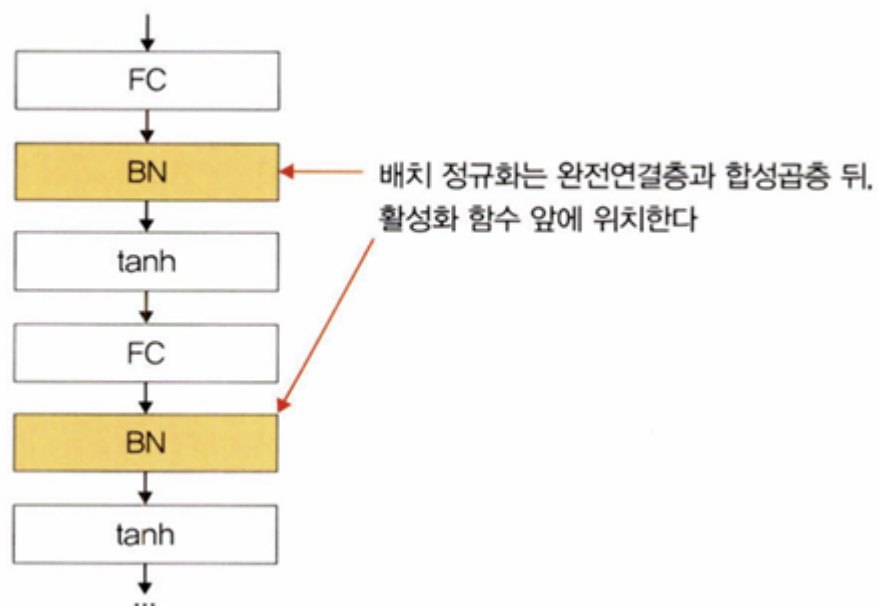
1. 배치 정규화가 적용되는 부분. BatchNorm1d에서 사용되는 파라미터는 특성 개수로 이전 계층의 출력 채널이 됨.

배치 정규화를 사용하는 이유는 은닉층에서 학습이 진행될 때마다 입력 분포가 변하면서 가중치가 엉뚱한 방향으로 갱신되는 문제가 종종 발생한다. 즉, 신경망의 층이 깊어질수록 학습할 때 가정했던 입력 분포가 변화하여 엉뚱한 학습이 진행될 수 있는데, 배치 정규화를 적용해서 입력 분포를 고르게 맞출 수 있다.

▼ 그림 8-39 배치 정규화



▼ 그림 8-40 배치 정규화 위치



코드 8-10 배치 정규화가 적용되지 않은 모델 선언

```
model = NormalNet()
print(model)

NormalNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
```

```

(1): ReLU()
(2): Linear(in_features=48, out_features=24, bias=True)
(3): ReLU()
(4): Linear(in_features=24, out_features=10, bias=True)
)
)

```

코드 8-11 배치 정규화가 적용된 모델 선언

```

model_bn = BNNet()
print(model_bn)

```

```

BNNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
    (1): BatchNorm1d(48, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Linear(in_features=48, out_features=24, bias=True)
    (4): BatchNorm1d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): Linear(in_features=24, out_features=10, bias=True)
  )
)

```

코드 8-12 데이터셋 메모리로 불러오기

```

batch_size = 512
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)

```

코드 8-13 옵티마이저, 손실 함수 지정

```

loss_fn = nn.CrossEntropyLoss()
opt = optim.SGD(model.parameters(), lr=0.01)
opt_bn = optim.SGD(model_bn.parameters(), lr=0.01)

```

코드 8-14 모델 학습

```

loss_arr = []
loss_bn_arr = []
max_epochs = 2

for epoch in range(max_epochs):
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        opt.zero_grad() # 배치 정규화가 적용되지 않은 모델 학습
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        opt.step()

```



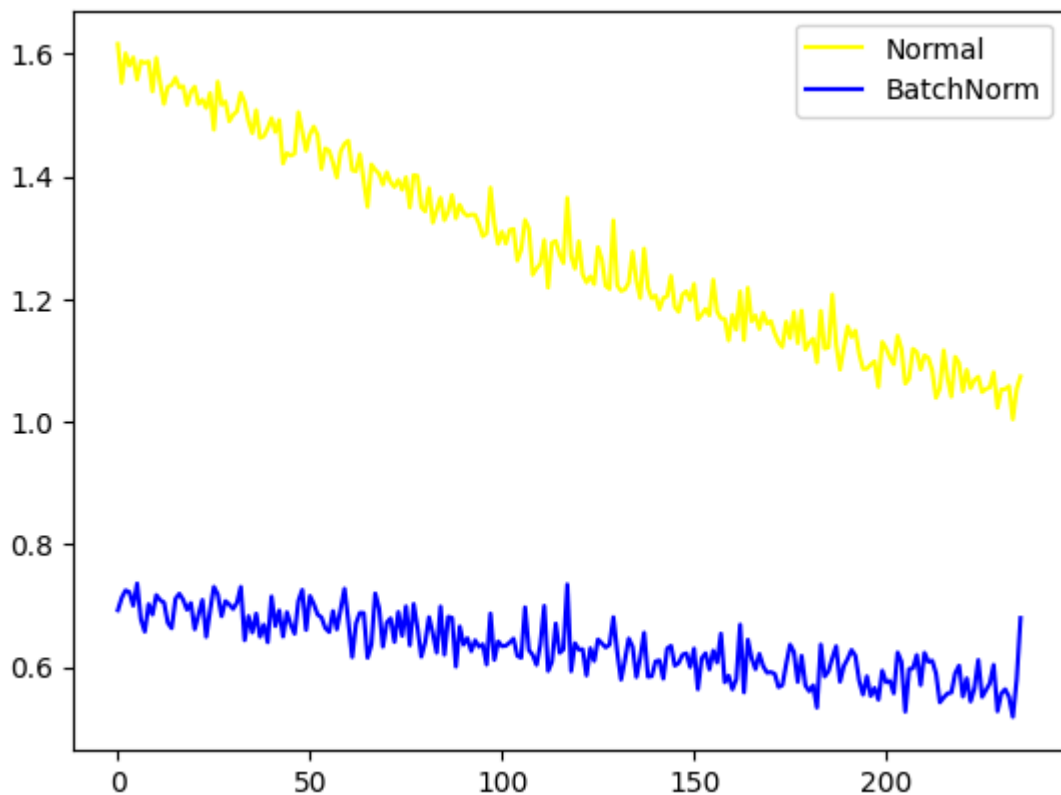
```

opt_bn.zero_grad() # 배치 정규화가 적용된 모델 학습
outputs_bn = model_bn(inputs)
loss_bn = loss_fn(outputs_bn, labels)
loss_bn.backward()
opt_bn.step()

loss_arr.append(loss.item())
loss_bn_arr.append(loss_bn.item())

plt.plot(loss_arr, 'yellow', label='Normal')
plt.plot(loss_bn_arr, 'blue', label='BatchNorm')
plt.legend()
plt.show()

```



그림들은 모델 학습 결과를 보여 준다.

배치 정규화가 적용된 모델의 경우 더 낮은 값으로 안정적인 범위 내에서 줄어들고 있는 것을 확인할 수 있다.

코드 8-15 데이터셋의 분포를 출력하기 위한 전처리

```

N = 50
noise = 0.3

x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1) # 1.
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1)) # 2.

```

```
x_test = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
y_test = x_test + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

1. 훈련 데이터셋이 -1~1 값을 갖도록 조정

```
x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
```

(a)

(b)

a. torch.linspace: 주어진 범위에서 균등한 값을 갖는 텐서를 만들기 위해 사용한다.

torch.linspace(-1, 1, N) 의미는 -1~1 범위에서 N개의 균등한 값을 갖는 텐서를 생성하겠다는 것

b. torch.unsqueeze: unsqueeze()는 차원을 늘리기 위해 사용한다. 따라서

torch.unsqueeze(torch.linspace(-1, 1, N), 1) 의미는 torch.linspace(-1, 1, N) 텐서의 첫 번째 자리에 차원을 증가시키겠다는 것

2. 훈련 데이터셋 값의 범위가 정규분포를 갖도록 조정한다.

```
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

(a) (b) (c)

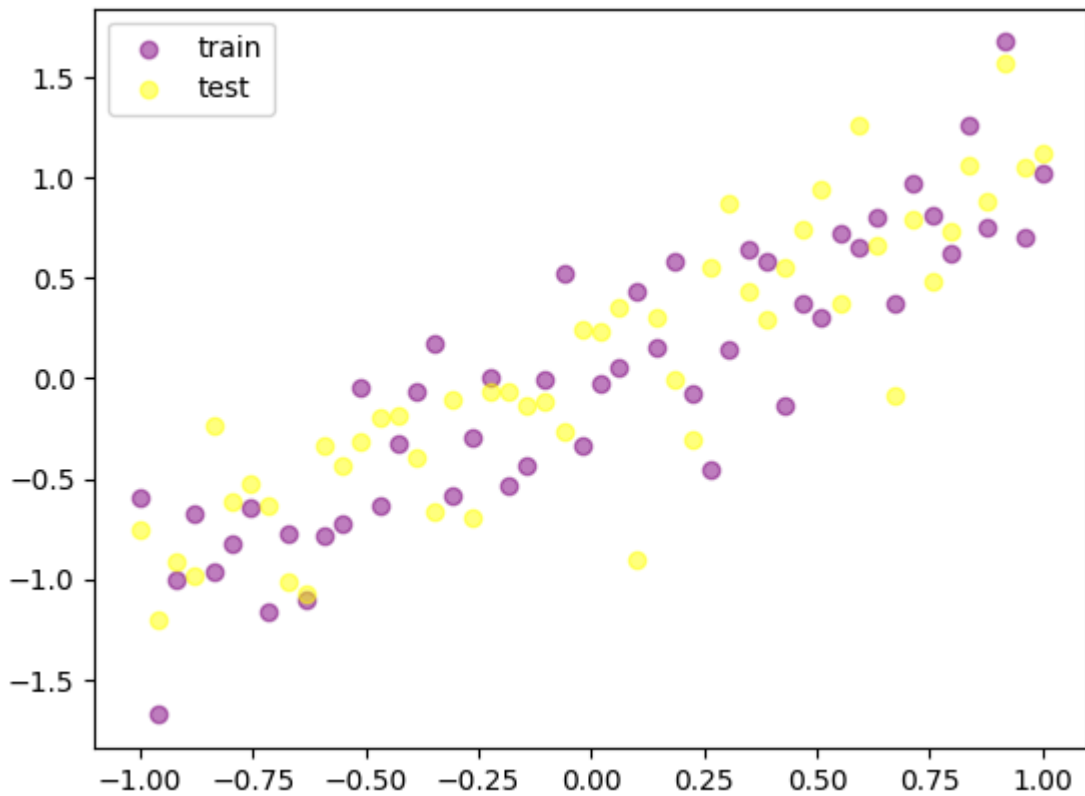
a. torch.normal: 정규분포로부터 무작위 표본 추출을 위해 사용한다. torch.normal(평균, 표준편차)를 의미하기 때문에 torch.zeros()는 평균, torch.ones()는 표준편차를 의미한다. 평균은 0, 표준편차는 1이 기본값이다.

b. torch.zeros: 0 값을 갖는 Nx1 텐서를 생성한다.

c. torch.ones: 1 값을 갖는 Nx1 텐서를 생성한다.

코드 8-16 데이터 분포를 그래프로 출력

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',
            alpha=0.5, label='train') # 1.
plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c='yellow',
            alpha=0.5, label='test')
plt.legend()
plt.show()
```



1. plt.scatter()는 데이터를 그래프상에 점으로 출력해서 데이터 분포를 확인하고자 할 때 사용한다.

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',
            alpha=0.5, label='train')
```

(a)
(b)
(c)
(d)
(e)

- a. x축에 위치할 데이터
- b. y축에 위치할 데이터
- c. c: 그래프로 출력되는 마커의 색상
- d. alpha: 마커에 대한 투명도를 조절. alpha=1이면 완전 불투명 상태
- e. label: 맷플롯립의 레전드와 같은 역할을 하지만 plt.legend()와 함께 사용되어야 함

코드 8-17 드롭아웃을 위한 모델 생성

```
N_h = 100
model = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1),
```

```
) # 드롭아웃이 적용되지 않은 모델
```

```
model_dropout = torch.nn.Sequential(  
    torch.nn.Linear(1, N_h),  
    torch.nn.Dropout(0.2), # 드롭아웃 적용  
    torch.nn.ReLU(),  
    torch.nn.Linear(N_h, N_h),  
    torch.nn.Dropout(0.2),  
    torch.nn.ReLU(),  
    torch.nn.Linear(N_h, 1),  
) # 드롭아웃이 적용된 모델
```

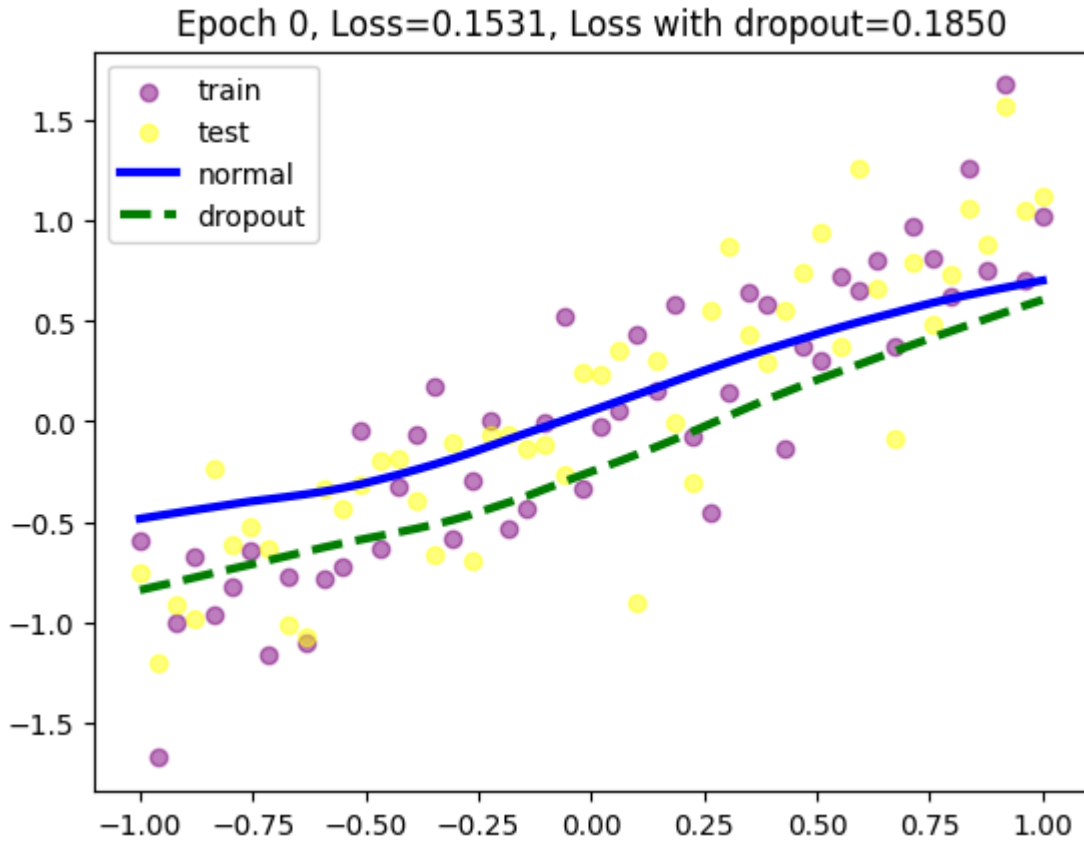
코드 8-18 옵티마이저와 손실 함수 지정

```
opt = torch.optim.Adam(model.parameters(), lr=0.01)  
opt_dropout = torch.optim.Adam(model_dropout.parameters(), lr=0.01)  
loss_fn = torch.nn.MSELoss()
```

코드 8-19 모델 학습

```
max_epochs = 1000  
for epoch in range(max_epochs):  
    pred = model(x_train) # 드롭아웃이 적용되지 않은 모델 학습  
    loss = loss_fn(pred, y_train)  
    opt.zero_grad()  
    loss.backward()  
    opt.step()  
  
    pred_dropout = model_dropout(x_train) # 드롭아웃이 적용된 모델 학습  
    loss_dropout = loss_fn(pred_dropout, y_train)  
    opt_dropout.zero_grad()  
    loss_dropout.backward()  
    opt_dropout.step()  
  
    if epoch % 50 == 0: # epoch를 50으로 나눈 나머지가 0이면 다음 진행  
        model.eval()  
        model_dropout.eval()  
  
        test_pred = model(x_test)  
        test_loss = loss_fn(test_pred, y_test)  
  
        test_pred_dropout = model_dropout(x_test)  
        test_loss_dropout = loss_fn(test_pred_dropout, y_test)  
  
    plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',  
                alpha=0.5, label='train')  
    plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c='yellow',  
                alpha=0.5, label='test')  
    plt.plot(x_test.data.numpy(), test_pred.data.numpy(), 'b-', lw=3,  
             label='normal') # 파란색 실선으로 x축은 테스트 데이터셋, y축은 드롭아웃이 적용되지 않은 모델  
    plt.plot(x_test.data.numpy(), test_pred_dropout.data.numpy(), 'g--', lw=3,  
             label='dropout') # 초록색 점선으로 x축은 테스트 데이터셋, y축은 드롭아웃이 적용된 모델의 결과  
    plt.title('Epoch %d, Loss=%0.4f, Loss with dropout=%0.4f' %  
              (epoch, test_loss, test_loss_dropout)) # 에포크, 드롭아웃이 적용되지 않은 모델의 오차,
```

```
plt.legend()
model.train()
model_dropout.train()
plt.pause(0.05)
```



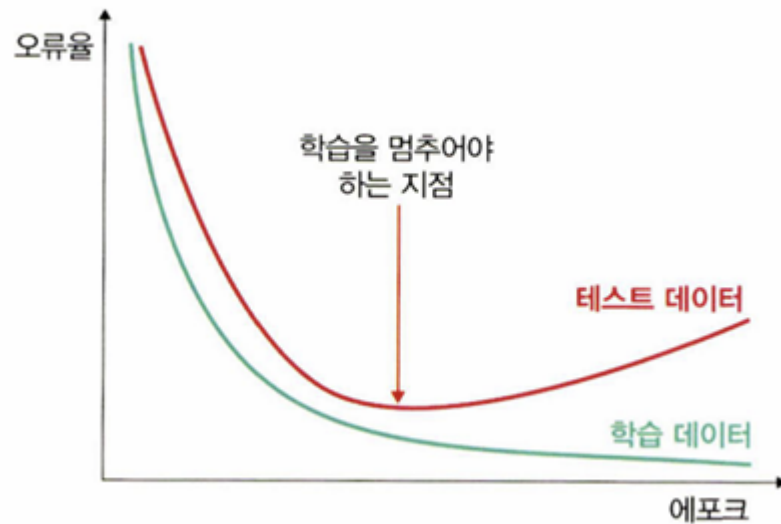
훈련 횟수가 늘어날수록 파란색 실선은 가장자리의 자주색 점들을 찾아가고 있다. 자주색 선은 훈련 데이터셋을 의미하며, 과적합 현상을 보이고 있다.



8.3.3 조기 종료를 이용한 성능 최적화

조기 종료: 과적합을 회피하는 규제 기법. 과적합이 발생하기 전까지 학습 오차와 검증 오차 모두 감소하지만, 과적합이 발생하면 훈련 데이터셋 오차는 감소하는 반면 검증 데이터셋 오차는 증가한다. 따라서 검증 데이터셋 오차가 증가하는 시점에서 학습을 멈추도록 조정한다.

▼ 그림 8-50 조기 종료



조기 종료는 학습을 언제 종료시킬지 결정할 뿐, 최고의 성능을 갖는 모델을 보장하지는 않는다.

코드 8-20 라이브러리 호출

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.models as models # 사전 학습된 모델 이용
from torchvision import transforms, datasets

import matplotlib
import matplotlib.pyplot as plt
import time
import argparse
from tqdm import tqdm
matplotlib.style.use('ggplot') # 출력 그래프에서 격자로 숫자 범위가 눈에 잘 띄도록 하는 스타일
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

코드 8-21 데이터셋 전처리

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
```

```
std=[0.229, 0.224, 0.225])
])
```

코드 8-22 데이터셋 가져오기

```
train_dataset = datasets.ImageFolder(
    root='/content/drive/MyDrive/chap08/data//archive/train',
    transform=train_transform
)
train_dataloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=32, shuffle=True,
)
val_dataset = datasets.ImageFolder(
    root=r'/content/drive/MyDrive/chap08/data//archive/test',
    transform=val_transform
)
val_dataloader = torch.utils.data.DataLoader(
    val_dataset, batch_size=32, shuffle=False,
)
```

코드 8-23 모델 생성

```
def resnet50(pretrained=True, requires_grad=False):
    model = models.resnet50(progress=True, pretrained=pretrained)
    if requires_grad == False: # 파라미터 고정하여 backward() 중에 기울기가 계산되지 않도록 함
        for param in model.parameters(): # requires_grad=False를 파라미터로 받았기 때문에 해당 구문이 실행
            param.requires_grad = False
    elif requires_grad == True: # 파라미터 값이 backward() 중에 기울기 계산에 반영됨
        for param in model.parameters():
            param.requires_grad = True
    model.fc = nn.Linear(2048, 2) # 마지막 분류를 위한 계층은 학습 진행
    return model
```

조기 종료와 함께 자주 사용되는 것으로 학습률 감소가 있다. 학습률에 대한 값을 고정시켜서 모델을 학습시키는 것이 아니라 학습이 진행되는 과정에서 학습률을 조금씩 낮추어 주는 성능 튜닝 기법 중 하나이다. 학습률 감소는 학습률 스케줄러라는 것을 이용하는데, 주어진 patience 횟수만큼 검증 데이터셋에 대한 오차 감소가 없으면 주어진 factor만큼 학습률을 감소시켜서 모델 학습 최적화가 가능하도록 도와준다.

코드 8-24 학습률 감소

```
class LRScheduler():
    def __init__(
        self, optimizer, patience=5, min_lr=1e-6, factor=0.5
    ):
        self.optimizer = optimizer
        self.patience = patience
        self.min_lr = min_lr
        self.factor = factor
        self.lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
```

```

        self.optimizer,
        mode='min',
        patience=self.patience,
        factor=self.factor,
        min_lr=self.min_lr,
        verbose=True
    ) # 1.
def __call__(self, val_loss):
    self.lr_scheduler.step(val_loss) # 2.

```

1. 학습 과정에서 모델 성능에 대한 개선이 없을 경우 학습률 값을 조절하여 모델의 개선을 유도하는 콜백 함수

```

torch.optim.lr_scheduler.ReduceLROnPlateau(self.optimizer, mode='min',
                                           (a) (b) (c)
                                           patience=self.patience, factor=self.factor, min_lr=self.min_lr, verbose=True)
                                           (d) (e) (f) (g)

```

2. 실제로 학습률을 업데이트함. 에포크 단위로 검증 데이터셋에 대한 오차를 받아서 이전 오차와 비교했을 때 차이가 없다면 학습률을 업데이트함

코드 8-25 조기 종료

```

class EarlyStopping():
    def __init__(self, patience=5, verbose=False, delta=0,
                 path='/content/drive/MyDrive/chap08/data/checkpoint.pt'):
        self.patience = patience # 1.
        self.verbose = verbose
        self.counter = 0
        self.best_score = None # 검증 데이터셋에 대한 오차 최적화값
        self.early_stop = False # 조기 종료 의미
        self.val_loss_min = np.Inf # 무한대 표현
        self.delta = delta # 2.
        self.path = path # 모델이 저장될 경로

    def __call__(self, val_loss, model):
        score = -val_loss
        if self.best_score is None:
            self.best_score = score
            self.save_checkpoint(val_loss, model)
        elif score < self.best_score + self.delta:
            self.counter += 1
            print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
            if self.counter >= self.patience:
                self.early_stop = True
            else:
                self.best_score = score
                self.save_checkpoint(val_loss, model)
                self.counter = 0

    def save_checkpoint(self, val_loss, model):
        if self.verbose:

```



```

        print(f'Validation loss decreased ({self.val_loss_min: .6f} --> {val_loss: .6f}). Saving
torch.save(model.state_dict(), self.path)
self.val_loss_min = val_loss

```

코드 8-26 인수 값 지정

```

parser = argparse.ArgumentParser()
parser.add_argument('--lr-scheduler', dest='lr_scheduler', action='store_true')
parser.add_argument('--early-stopping', dest='early_stopping', action='store_true')
args = vars(parser.parse_args([]))

```

코드 8-27 사전 훈련된 모델의 파라미터 확인

```

print(f"Computation device: {device}\n")
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT).to(device)
total_params = sum(p.numel() for p in model.parameters())
print(f"{total_params:,} total parameters.")
total_trainable_params = sum(
    p.numel() for p in model.parameters() if p.requires_grad
)
print(f"{total_trainable_params:,} training parameters.")

```

코드 8-28 옵티마이저와 손실 함수 지정

```

lr = 0.001
epochs = 100
optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.CrossEntropyLoss()

```

코드 8-29 오차, 정확도 및 모델의 이름에 대한 문자열

```

loss_plot_name = 'loss'
act_plot_name = 'accuracy'
model_name = 'model'

```

코드 8-30 오차, 정확도 및 모델의 이름에 대한 문자열

```

if args['lr_scheduler']:
    print('INFO: Initializing learning rate scheduler')
    lr_scheduler = LRScheduler(optimizer)
    loss_plot_name = 'lrs_loss'
    acc_plot_name = 'lrs_accuracy'
    model_name = 'lrs_model'
if args['early_stopping']:
    print('INFO: Initializing early stopping')
    early_stopping = EarlyStopping()
    loss_plot_name = 'es_loss'

```

```
acc_plot_name = 'es_accuracy'
model_name = 'es_model'
```

코드 8-31 모델 학습 함수

```
def training(model, train_dataloader, train_dataset, optimizer, criterion):
    print('Training')
    model.train()
    train_running_loss = 0.0
    train_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(train_dataloader), total=int(len(train_dataset)/
                                                             train_dataloader.batch_size))

    for i, data in prog_bar:
        counter += 1
        data, target = data[0].to(device), data[1].to(device)
        total += target.size(0)
        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, target)
        train_running_loss += loss.item()
        _, preds = torch.max(outputs.data, 1)
        train_running_correct += (preds == target).sum().item()
        loss.backward()
        optimizer.step()

    train_loss = train_running_loss / counter
    train_accuracy = 100. * train_running_correct / total
    return train_loss, train_accuracy
```

코드 8-32 모델 검증 함수

```
def validate(model, test_dataloader, val_dataset, criterion):
    print('Validating')
    model.eval()
    val_running_loss = 0.0
    val_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(test_dataloader), total=int(len(val_dataset)/
                                                             test_dataloader.batch_size))

    with torch.no_grad():
        for i, data in prog_bar:
            counter += 1
            data, target = data[0].to(device), data[1].to(device)
            total += target.size(0)
            outputs = model(data)
            loss = criterion(outputs, target)

            val_running_loss += loss.item()
            _, preds = torch.max(outputs.data, 1)
            val_running_correct += (preds == target).sum().item()
```

```

val_loss = val_running_loss / counter
val_accuracy = 100. * val_running_correct / total
return val_loss, val_accuracy

```

코드 8-33 모델 학습

```

train_loss, train_accuracy = [], []

val_loss, val_accuracy = [], []

start = time.time()
for epoch in range(epochs):
    print(f"Epoch {epoch+1} of {epochs}")
    train_epoch_loss, train_epoch_accuracy = training(model, train_dataloader, train_dataset, op
    val_epoch_loss, val_epoch_accuracy = validate(model, val_dataloader, val_dataset, criterion)
    train_loss.append(train_epoch_loss)
    train_accuracy.append(train_epoch_accuracy)
    val_loss.append(val_epoch_loss)
    val_accuracy.append(val_epoch_accuracy)
    if args['lr_scheduler']:
        lr_scheduler(val_epoch_loss)
    if args['early_stopping']:
        early_stopping(val_epoch_loss, model)
        if early_stopping.early_stop:
            break
    print(f"Train Loss: {train_epoch_loss:.4f}, Train Acc: {train_epoch_accuracy:.2f}")
    print(f"Val Loss: {val_epoch_loss:.4f}, Val Acc: {val_epoch_accuracy:.2f}")
end = time.time()
print(f"Training time: {(end-start)/60:.3f} minutes")

```

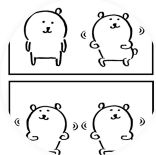
코드 8-34 모델 학습 결과 출력

```

print('Saving loss and accuracy plots...')
plt.figure(figsize=(10,7))
plt.plot(train_accuracy, color='green', label='train accuracy')
plt.plot(val_accuracy, color = 'blue', label='validation accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.savefig(f"../chap08/img/{acc_plot_name}.png")
plt.show()
plt.figure(figsize=(10,7))
plt.plot(train_loss, color='orange', label='train loss')
plt.plot(val_loss, color='red', label='validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.savefig(f"../chap08/img/{loss_plot_name}.png")
plt.show()

print('Saving model...')
torch.save(model.state_dict(), f"../chap08/model/{model_name}.pth")
print('TRAINING COMPLETE')

```



카사바

안녕하세요 :)



이전 포스트

Week12_예습과제

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate